

Phần 1 - Phân tích logic: Thách thức của Microservices và giới hạn của Unit Test

1. Thách thức kiểm thử của Microservices so với Monolith

Trong kiến trúc nguyên khối (Monolith), toàn bộ ứng dụng chia sẻ chung một cơ sở dữ liệu và giao tiếp với nhau qua các lời gọi hàm cục bộ (in-memory function calls). Tuy nhiên, khi chuyển sang Microservices, chúng ta phải đối mặt với các thách thức hoàn toàn mới:

- **Độ trễ và rủi ro mạng:** Giao tiếp giữa BookingService và UserService giờ đây là qua mạng (REST/gRPC). Các lỗi như timeout, rớt mạng, hoặc phản hồi chậm trở thành câu chuyện thường ngày.
- **Dữ liệu phân tán:** Mỗi service có một Database riêng. Việc đảm bảo tính nhất quán của dữ liệu (ví dụ: tạo Booking thành công nhưng không tìm thấy User) khó kiểm tra hơn rất nhiều.
- **Sự phụ thuộc chéo (Dependency Hell):** Để test BookingService, bạn cần UserService và NotificationService hoạt động ổn định. Nếu một service thay đổi cấu trúc API (contract), các service khác gọi đến nó có thể sụp đổ ngay lập tức.
- **Quản lý môi trường:** Việc dựng lên toàn bộ hệ thống gồm hàng chục service cùng các dependencies (Kafka, Redis, PostgreSQL) để test trên máy cá nhân là cực kỳ tốn tài nguyên và phức tạp.

2. Tại sao Unit Test là chưa đủ?

Unit Test cực kỳ xuất sắc trong việc kiểm tra logic nội bộ của từng class hoặc method (ví dụ: thuật toán tính giá tiền trong BookingService). Tuy nhiên, bản chất của Unit Test là **cô lập (isolation)** bằng cách sử dụng Mock objects.

- Khi bạn mock UserService trong Unit Test của BookingService, bạn chỉ đang test dựa trên *giả định* của bạn về cách UserService hoạt động.
- Nếu đội ngũ phát triển UserService thay đổi kiểu dữ liệu trả về (ví dụ: đổi userId từ Long sang String), Unit Test của BookingService vẫn sẽ **Pass** (màu xanh), nhưng khi chạy thực tế trên production, hệ thống sẽ **Crash**.
- Do đó, Unit Test không thể phát hiện các lỗi liên quan đến giao tiếp qua mạng, lỗi sai lệch cấu trúc API, hoặc lỗi cấu hình kết nối Database.

Phần 2 - Thực thi: Báo cáo Chiến lược Kiểm thử Hệ thống Booking Microservices

1. Mô hình kiểm thử: Kim tự tháp Microservices (Microservices Test Pyramid)

Thay vì sử dụng kim tự tháp truyền thống, chiến lược này áp dụng biến thể phù hợp cho Microservices, tập trung vào việc giảm thiểu End-to-End (E2E) và tăng cường Contract Testing.

- **Unit Tests (Khoảng 60%):** Nền tảng của kim tự tháp. Kiểm tra các logic nghiệp vụ thuần túy (tính toán giá, parse dữ liệu, validate đầu vào) bên trong từng service. Chạy cực nhanh, phản hồi ngay lập tức.
- **Integration Tests (Khoảng 20%):** Kiểm tra sự tích hợp giữa code và các thành phần ngoại vi *thuộc quyền kiểm soát của service đó* (ví dụ: kết nối Database, cấu hình Spring Boot, gọi Redis cache).
- **Contract Tests (Khoảng 15%):** Rất quan trọng trong Microservices. Đây là lớp thay thế một phần lớn E2E Test. Nó đảm bảo BookingService (Consumer) và UserService (Provider) thống nhất với nhau về định dạng Request/Response mà không cần phải chạy cả hai service cùng lúc.
- **End-to-End Tests (Khoảng 5%):** Khối lượng ít nhất, chỉ dùng để kiểm tra các luồng nghiệp vụ quan trọng nhất (Happy paths) như: Đăng nhập -> Đặt lịch -> Nhận Email. Tốn kém, chạy chậm và dễ bị "flaky" (lúc pass lúc fail do mạng).

2. Công cụ và Kỹ thuật đề xuất

Dựa trên hệ sinh thái Java/Spring Boot phổ biến, các công cụ được lựa chọn bao gồm:

- **Tầng Unit Test:**
 - **JUnit 5 & Mockito:** Tiêu chuẩn vàng để viết test và mock các dependencies nội bộ.
 - **AssertJ:** Viết các câu lệnh assert dễ đọc và theo luồng (fluent API).
- **Tầng Integration Test:**
 - **Spring Boot Test Slices:** Sử dụng `@DataJpaTest` để test lớp Repository với cấu hình DB in-memory hoặc Testcontainers; `@WebMvcTest` để test tầng Controller mà không cần dựng toàn bộ context Spring.
 - **Testcontainers:** Thay vì dùng H2 in-memory (dễ sai lệch cú pháp so với DB thật), Testcontainers sẽ dựng các container Docker chứa DB thật (PostgreSQL/MySQL) ngay trong lúc chạy test để đảm bảo môi trường test giống hệt production.
- **Tầng Contract / API Test:**
 - **REST Assured:** Khai báo và verify các API endpoints một cách rõ ràng.
- **Tầng End-to-End (E2E) Test:**

- **Cypress (hoặc Playwright):** Viết script mô phỏng tương tác của người dùng trên UI (Web/App) đi qua tất cả các microservices thực tế.

3. Chiến lược Testing Coverage với JaCoCo và Quality Gates

- **Cấu hình JaCoCo:** Tích hợp JaCoCo plugin vào pom.xml hoặc build.gradle của từng microservice. Khi chạy build, JaCoCo sẽ quét qua mã nguồn và thống kê mã nào đã được Unit/Integration test bao phủ.
- **Thiết lập Quality Gates:** Thiết lập rule chặt chẽ: minimum line coverage = 80% và minimum branch coverage = 70%.
- **Lý do áp dụng:** Quality Gates đóng vai trò là "người gác cổng" tự động. Nếu một Developer push code lên thiếu test (ví dụ: coverage bị tụt xuống 79%), quá trình Build sẽ **báo lỗi (Fail) ngay lập tức** và từ chối merge code. Điều này triệt tiêu tình trạng nợ kỹ thuật (technical debt) và ép buộc văn hóa viết test trong đội ngũ.

4. Giải pháp kiểm thử giao tiếp giữa các Services (Inter-service Testing)

Để tránh rủi ro khi UserService, BookingService và NotificationService giao tiếp với nhau, ta áp dụng 2 chiến lược:

- **Consumer-Driven Contract Testing (CDC) với Pact:** * BookingService (người tiêu thụ) sẽ viết một "bản hợp đồng" (Contract) mô tả chính xác nó cần UserService trả về một JSON như thế nào khi tìm kiếm User.
 - Contract này được đẩy lên một Broker (Pact Broker).
 - Trong CI/CD của UserService (người cung cấp), hệ thống sẽ tự động kéo Contract này về và chạy test dựa trên đó. Nếu UserService thay đổi code làm sai lệch Contract, CI của UserService sẽ gãy. Cách này đảm bảo API luôn tương thích mà không cần dựng môi trường phức tạp.
- **Testcontainers cho các ngoại vi hệ thống:** Nếu NotificationService cần đẩy event vào RabbitMQ/Kafka, Integration Test sẽ dùng Testcontainers để spin-up một Kafka container thực, gửi tin nhắn giả và verify xem tin nhắn có được consume đúng không.

5. Tích hợp vào quy trình CI/CD

Tất cả các bài kiểm thử trên vô nghĩa nếu phải chạy bằng tay. Pipeline CI/CD (trên GitLab CI, GitHub Actions hoặc Jenkins) sẽ được thiết kế tự động hóa theo các bước (Fail fast - Lỗi ở đâu dừng ở đó):

1. **Code Commit & Build:** Lập trình viên push code. CI tự động checkout nhánh, compile code.

2. **Unit Tests & Coverage Check:** Chạy toàn bộ Unit Tests. Kích hoạt JaCoCo. Nếu Quality Gate không đạt chuẩn 80/70, dừng pipeline và báo fail.
3. **Integration Tests:** Spin-up Docker Testcontainers (Database thật) để chạy các test liên quan tới Repository/Services.
4. **Contract Tests:** Xác thực hợp đồng API (Pact) với các dịch vụ khác thông qua Pact Broker.
5. **Build Image:** Nếu mọi test thành công, đóng gói ứng dụng thành Docker Image và push lên Registry.
6. **Deploy to Staging & E2E Test:** Deploy Image lên môi trường Staging. Kích hoạt Cypress/Playwright để chạy E2E UI testing mô phỏng người dùng thật.
7. **Production Readiness:** Toàn bộ xanh (Passed) thì code mới đủ điều kiện để release ra Production.